

Distributed Systems – Labs

Java - Programming with HTTP for the Internet

Learning Outcomes:

1. Be able to implement a simple HTTP client/server application in Java.

Tasks

1. Building a simple HTTP server

We will use a class called `WebServer` to support the HTTP/ 1.0 protocol. The server will use a `ClientHandler` class to handle a client. The server will be limited to handling only GET requests. However, this will be adequate to illustrate the basic server elements needed. Support of other methods can be easily added.

Extract T1/ `WebServer.java`, `ClientHandler.java` and `HTTPClient.java` files.

Examine the source code. The `ServerSocket` class is the foundation of the server. Its `accept` method will block until a request is made. When this happens, a new thread based on the `ClientHandler` class will be started.

Mac users may encounter an error when using port 80. Use port 3000 or 8080 instead.

The `ClientHandler` class, shown next, implements the `Runnable` interface. Its constructor is passed to the socket representing the client. When the thread starts, the `run` method executes. The method displays, starting and terminating messages. The actual work is performed in the `handleRequest` method. The `handleRequest` method uses the input and output streams to communicate with the server. In addition, it determines what request was made and then processes that request.

In the code that follows, the input and output streams are created and the First line of the request is read. The `StringTokenizer` class is used to token this line. When the `nextToken` method is invoked, it returns the first word of the line, which should correspond to an HTTP method:

When the server starts, it will display the following:

```
Webserver Started  
Waiting for client request
```

Building a simple HTTP client

We will use the following `HTTPClient` class to access our HTTP server. In its constructor, a socket connecting to the server is created. The `Socket` class's `getInputStream` and `getOutputStream` return input and output streams for the socket, respectively. The `sendGet` method is called, which sends a request to the server. The `getResponse` method returns the response, which is then displayed:

Extract T1/ `HTTPClient.java` file and examine the source code.

When the client application executes, we get the following output reflecting the server's response:

```
HTTP Client Started
HTTP/1.0 200 OK
Server: WebServer
Content-Type: text/html
Content-Length: 86
```

```
<html><h1>WebServer Home Page.... </h1><br><b>Welcome to
my web server!</b><BR></html>
```

If we enter the URL `http://127.0.0.1/index` in a browser, the browser will display the page:

```
WebServer Home Page....
Welcome to my web server!
```

2. Using `HTTPServer` and `HttpURLConnection` classes

Extract T2/`MyHTTPServer.java` and `HttpURLConnectionExample.java` files and examine the source code.

The `HTTPServer` class provides a set of features to support a simple HTTP server. Many of the tasks that we had to perform manually with our previous server example are simplified with this server. The interaction between a client and server is referred to as an exchange. This and other supporting classes and interfaces are members of the `com.sun.net.httpserver` package. Most `com.sun.*` packages in the JDK are for internal use, but a few are supported for external use (including `HTTPServer`) They are normally included with most IDEs, but See Note 1 below.

The `MyHTTPServer` class implements a simple server using the `HTTPServer` class. An instance of the `HttpServer` class is created using the localhost and port 80 (3000 or 8080 on a Mac). The `createContext` method associates the `/index` path with an instance of the `IndexHandler` class. This handler will process the request. The `start` method starts the server. The server will continue running, processing multiple requests until it is manually stopped.

The `URLConnection` class provides a convenient technique to access a web server. With this class, we can connect to a site, make a request, and access the response headers and the response message.

The `URLConnectionExample` class is defined as follows. A `sendGet` method supports transmitting the GET method request to the server. The `URLConnectionExample` class supports other HTTP methods. For this example, only GET method is used.

The `sendGet` method implementation: A Google query (<http://www.google.com/search?q=java+sdk&ie=utf-8&oe=utf-8>) is used to illustrate the process where we search for “java sdk”. The latter part of the query, `&ie=utf-8&oe=utf-8`, is additional information attached to the query by the Google search engine.

URL encoding translates special characters from the URL to a representation that adheres to the spec and can be correctly understood and interpreted. When a URL is formed, a specific URL format needs to be used. In Java, we can perform URL encoding using the `URLEncoder` class. We need to encode only the query portion of the URI. E.g.

```
String urlQuery = "http://www.google.com/search?q=";  
String userQuery = "Java SE";  
String urlEncoded = urlQuery + URLEncoder.encode(  
    userQuery, "UTF-8");
```

When `URLConnectionExample` executes, you will get output as follows. Due to the length of the output, it has been truncated:

```
Sent Http GET request  
http://www.google.com/search?q=java+sdk  
URI created:  
https://docs.oracle.com/en/java/javase/17/security/index.html#Java-Platform%2C-Standard-Edition  
URL from URI:  
https://docs.oracle.com/en/java/javase/17/security/index.html#Java-Platform%2C-Standard-Edition  
Response Code: 200  
Request Headers  
  
response: <!DOCTYPE html  SYSTEM "about:legacy-compat">...
```

You can see the full output if you use this query in a browser.

Note 1:

Just to note - from

<https://openjdk.org/jeps/403>

Exported `com.sun` APIs

Most `com.sun.*` packages in the JDK are for internal use, but a few are supported for external use. These supported packages were exported in JDK 9 and will continue to be exported, so you can continue to program against their public APIs. They will, however, no longer be open. Examples include

- The Compiler Tree API in the [jdk.compiler](#) module,
- The HTTP Server API in the [jdk.httpserver](#) module,
- The SCTP API in the [jdk.sctp](#) module, and
- JDK-specific extensions to the NIO API in the `com.sun.nio.file` package of the `jdk.unsupported` module.

Java 18's Simple Web Server

Java 18's Simple Web Server is a minimal HTTP static file server that was added in JEP 408 to the `jdk.httpserver` module. It is useful for prototyping, ad hoc coding, and testing.

You can read the details how to use it at:

<https://openjdk.org/jeps/408>

<https://blogs.oracle.com/javamagazine/post/java-18-simple-web-server>

It serves a single directory hierarchy; dynamic content and other HTTP versions are not supported.

The server supports only the HEAD and GET request methods; any other requests receive either a 501 - Not Implemented or a 405 - Not Allowed response.

References

Chapter 4: Richard M Reese, Learning Network Programming with Java, 2015 Packt Publishing

<https://openjdk.org/jeps/408>

<https://blogs.oracle.com/javamagazine/post/java-18-simple-web-server>